

Zadanie D – Owocowy ogród

Punktów do uzyskania: 10

Zadanie implementuje klasy obsługujące owocowy ogród.

- Ogólny opis klas:
 - Ogród
 - Jest odwzorowany w klasie GARDEN_CLASS.
 - Może obejmować dowolną ilość drzew.
 - Definicję klasy musi zawierać pliku garden.hpp, natomiast kompilacji podlega plik garden.cpp.
 - Drzewo
 - Jest odwzorowane w klasie TREE_CLASS.
 - Posiada unikalny nieujemny numer całkowity.
 - Ma określoną nieujemną wysokość.
 - Może obejmować dowolną nieujemną ilość gałęzi.
 - Definicję klasy musi zawierać pliku tree.hpp, natomiast kompilacji podlega plik tree.cpp.
 - Gałąź
 - Jest odwzorowana w klasie BRANCH_CLASS.
 - Ma określoną długość.
 - Może obejmować dowolną ilość owoców.
 - Definicję klasy musi zawierać pliku branch.hpp, natomiast kompilacji podlega plik branch.cpp.
 - Owoc
 - Jest odwzorowany w klasie wie FRUIT_CLASS.
 - Ma określoną wagę.
 - Definicję klasy musi zawierać pliku fruit.hpp, natomiast kompilacji podlega plik fruit.cpp.
 - Wartości dotąd użyte oraz wymienione poniżej zaw-sze mieszczą się w zakresie typu **unsigned int**:
 - Ilość drzew.
 - Ilość wszystkich gałęzi.
 - Ilość wszystkich owoców.
 - Suma wag wszystkich owoców.
 - Numer drzewa.
 - Wysokość drzewa.
 - Długość gałęzi.
 - Waga owocu.
- Wymagane metody klasy GARDEN_CLASS.
 - unsigned int** getTreesTotal (**void**)
Zwraca ilość drzew w ogrodzie.
 - unsigned int** getBranchesTotal (**void**)
Zwraca ilość gałęzi w ogrodzie.
 - unsigned int** getFruitsTotal (**void**)
Zwraca ilość owoców ogrodzie.
 - unsigned int** getWeightsTotal (**void**)
Zwraca wagę owoców w ogrodzie.
 - void** plantTree (**void**)
Dodaje nowe drzewo o najniższym możliwym iden-tyfikatorze, zerowej wysokości i zerowej ilości ga-łęzi.

- void** extractTree (**unsigned int**)
Usuwa drzewo o numerze danym w argumencie o ile istnieje.
 - void** growthGarden (**void**)
Dojrzewa każde drzewo według metody growth-Tree dla klasy TREE_CLASS.
 - void** fadeGarden (**void**)
Więdnie każde drzewo według metody fadeTree dla klasy TREE_CLASS.
 - void** harvestGarden (**unsigned int**)
Zrywa wszystkie owoce o wadze nie mniejszej od danej argumentem zgodnie z metodą harvestTree dla klasy TREE_CLASS.
 - TREE_CLASS* getTreePointer (**unsigned int**)
Zwraca wskaźnik do drzewa o numerze danym argu-mentem lub wartość NULL, gdy wymaganego drzewa brak.
 - void** cloneTree (**unsigned int**)
 - Dodaje nowe drzewo o najniższym możliwym nu-merze będące klonem drzewa o numerze danym argumentem - o ile istnieje.
 - W implementacji możliwe użycie wyłącznie kons-truktorów kopiujących.
- Wymagane metody klasy TREE_CLASS.
 - unsigned int** getBranchesTotal (**void**)
Zwraca ilość gałęzi drzewa.
 - unsigned int** getFruitsTotal (**void**)
Zwraca ilość owoców drzewa.
 - unsigned int** getWeightsTotal (**void**)
Zwraca sumę wag wszystkich owoców drzewa.
 - unsigned int** getNumber (**void**)
Zwraca numer drzewa.
 - unsigned int** getHeight (**void**)
Zwraca wysokość drzewa.
 - void** growthTree (**void**)
 - Zwiększa wysokość drzewa o wartość 1.
 - Każdą gałąź drzewa dojrzewa według metody growthBranch dla klasy BRANCH_CLASS.
 - Osiągnięcie wysokości drzewa równej każdej do-datniej wielokrotności liczby 3 powoduje powsta-nie nowej gałęzi o zerowej długości i zerowej ilo-ści owoców.
 - Na określonej wysokości drzewa może rosnąć jedna gałąź.
 - void** fadeTree (**void**)
 - Zmniejsza wysokość drzewa o wartość 1.
 - Każda gałąź drzewa więdnie według metody fadeBranch dla klasy BRANCH_CLASS.
 - Każda gałąź powyżej zmniejszonej wysokości zo-staje usunięta.
 - Drzewo o zerowej wysokości nie zostaje usunięte i może dojrzewać.

- void** harvestTree (**unsigned int**)
Wszystkie owoce wszystkich gałęzi o wadze nie mniejszej od danej argumentem zostają zerwane zgodnie z metodą harvestBranch dla klasy BRANCH_CLASS.
 - void** cutTree (**unsigned int**)
 - Przycina drzewo do wysokości danej argumentem.
 - Wszystkie gałęzie powyżej zmniejszonej wysoko-ści zostają usunięte.
 - void** cloneBranch (BRANCH_CLASS*)
 - Pierwsza gałąź (licząc od najniższej wysokości drzewa) o zerowej długości zostaje zastąpiona klonem gałęzi wskazanej argumentem.
 - Uwaga: w implementacji możliwe użycie wyłącz-nie konstruktorów kopiujących.
 - GARDEN_CLASS* getGardenPointer (**void**)
Zwraca wskaźnik do ogrodu, w którym rośnie dane drzewo, lub wartość NULL gdy drzewo istnieje bez ogrodu.
 - BRANCH_CLASS* getBranchPointer (**unsigned int**)
Zwraca wskaźnik do gałęzi wyrastającej na wysoko-ści danej argumentem lub wartość NULL, gdy brak gałęzi na zadanej wysokości.
- Wymagane metody klasy BRANCH_CLASS.
 - unsigned int** getFruitsTotal (**void**)
Zwraca ilość owoców na gałęzi.
 - unsigned int** getWeightsTotal (**void**)
Zwraca sumaryczną wagę owoców na gałęzi.
 - unsigned int** getHeight (**void**)
Zwraca wysokość na której rośnie gałąź.
 - unsigned int** getLength (**void**)
Zwraca długość gałęzi.
 - void** growthBranch (**void**)
 - Zwiększa długość gałęzi o wartość 1.
 - Każda dotychczasowy owoc zwiększa wagę o 1.
 - Osiągnięcie długości równej każdej dodatniej wie-lokrotności liczby 2 powoduje powstanie nowego owocu o zerowej wadze.
 - Na określonej długości gałęzi może rosnąć jeden owoc.
 - void** fadeBranch (**void**)
 - Zmniejsza długość gałęzi o wartość 1.
 - Zmniejsza wagę każdego owocu o wartość 1.
 - Usuwa każdy owoc powyżej zmniejszonej długo-ści.
 - Gałąź o zerowej długości nie zostaje usunięta i może dojrzewać.
 - void** harvestBranch (**unsigned int**)
Zrywa wszystkie owoce o wadze nie mniejszej od danej argumentem zgodnie z metodą pluckFruit dla klasy FRUIT_CLASS.

- void** cutBranch (**unsigned int**)
 - Przycina gałąź do długości danej argumentem.
 - Usuwa wszystkie owoce na długości powyżej zmniejszonej.
 - FRUIT_CLASS* getFruitPointer (**unsigned int**)
Zwraca wskaźnik do owocu wyrastającego w odle-głości danej argumentem lub wartość NULL, gdy owocu w zadanej odległości brak.
 - TREE_CLASS* getTreePointer (**void**)
Zwraca wskaźnik do drzewa obejmującego daną gałąź.
- Wymagane metody klasy FRUIT_CLASS.
 - unsigned int** getLength (**void**)
Zwraca długość na jakiej rośnie owoc.
 - unsigned int** getWeight (**void**)
Zwraca wagę owocu.
 - void** growthFruit (**void**)
Zwiększa wagę owocu o wartość 1.
 - void** fadeFruit (**void**)
Zmniejsza wagę owocu o wartość 1.
 - void** pluckFruit (**void**)
Zeruje wagę owocu nie usuwając go.
 - BRANCH_CLASS* getBranchPointer (**void**)
Zwraca wskaźnik do gałęzi obejmującej dany owoc.
- Dodatkowe uwarunkowania:
 - Wszystkie pola muszą mieć hermetyzację **private**.
 - Wszystkie metody muszą mieć hermetyzację **public**.
 - Zabronionymi są:
 - Zmienne globalne.
 - Nawiasy kwadratowe i ich równoważniki.
 - Słowa **friend** i **static**.
 - Podprogramy niestanowiące metod, czyli ogólnie podprogramy zewnętrzne.
 - Własne zewnętrzne klasy inne niż cztery wyma-gane.
 - Kontenery.
 - Identyfikatory zaczynające się znakiem podkreśle-nia.
 - Konstruktory oraz destruktory muszą zapewnić zwrot przydzielonej pamięci oraz gwarantować użycie operatora **delete** wyłącznie dla argumentów usta-lonych przez użycie operatora **new**.
 - Brak wygórowanych wymagań czasowych poza zaka-zem stosowania algorytmów o złożoności liniowej w sytuacji, gdy istnieje rozwiązanie o złożoności sta-łej, czy zakazem stosowania algorytmów o złożoności kwadratowej, gdy istnieje rozwiązanie o złożoności li-niowej.
 - Każdy wysyłane na BeCę plik musi zawierać w pierw-szej linii komentarz z imieniem i nazwiskiem autora, zaś wysyłane pliki .hpp oraz .cpp muszą być spako-wane do jednego pliku programem zip..